
Robotics Practicals

Ros2

Yasmine Tligui | 341215

Ismail Filali | 345459

Hajroun Zaynab | 346235

Section 1

Main goal :

In the first session, we were introduced to the foundational concepts of ROS2. The main goal was to familiarize ourselves with the ROS2 project structure and core components such as nodes, topics, publishers, subscribers, and the ROS2 build system. We learnt about many different components :

- **Nodes** : small programs doing specific tasks
In the TP, we created several nodes : a minimal node to test our setup, a publisher, and a subscriber. The publisher sent messages every 0.5 seconds to a topic named `topic`, and the subscriber received and displayed those messages using a simple callback. Each node had a clear job either sending data or reacting to it.
- **Topics** : channels used to send and receive data
We saw this when running the talker and listener example, and then again in our heat index system. The temperature and humidity data were sent through separate topics that we defined ourselves.
- **Publishers** : nodes that send data to a topic
In the TP, we created two publishers : one for temperature and one for humidity. They published random values every 5 seconds to simulate real sensor data.
- **Subscribers** : nodes that receive data from a topic
We also built a subscriber node that listened to both temperature and humidity topics. It used this data to calculate the heat index and showed a warning if the value was too high.
- **Server / Client** :
We didn't use services in this session, but they'll be useful later when we want one node to send a request and wait for an answer, like asking a robot to perform an action.

During the practical mini project, we created two publishers (simulating temperature and humidity sensors) and a subscriber that calculated the heat index based on both data streams. If the heat index passed a dangerous threshold, the system would log a warning message. This part made us become more autonomous and led to a better overview of how to use what we just discovered.

Issues we faced and what we learnt

During the session, we faced several small issues that helped us better understand how ROS2 works. Most of them were related to small mistakes or missing steps, but each one taught us something important about the development and debugging process in ROS2. Here are the main problems we ran into :

- **Forgetting to make files executable**
After creating the Python files for the nodes (like `minimal-node.py`, `publisher.py`, and `subscriber.py`), we forgot to run `chmod +x` on them. Without that, ROS2 couldn't run them even if everything else was correct. Even though the node was declared in `setup.py`, we forgot to run `chmod +x`, which is still needed to make the script runnable by ROS2.
- **Not rebuilding the workspace after changes**
Sometimes we modified files or added new ones but forgot to run `colcon build` again. As a result, changes weren't taken into account.
- **Forgetting to source the setup file after building**
After building with `colcon build`, we needed to run `source install/setup.bash`. If we didn't do it, ROS2 couldn't find the new executables, and running `ros2 run` failed.
- **Mixing up executable name, file name, and node name**
It was a bit confusing at first to understand the difference between : executable name (used in `ros2 run`), file name (the `.py` script), and node name (given in the constructor of the node class). Sometimes we used the wrong name in the command.
- **Confusion with topic names**
We used the wrong topic name in one of the subscribers and couldn't receive any messages. Using `ros2 topic list` and `ros2 topic echo` helped us realize the mistake.

- **Not seeing any output from the node**

In early tests, we commented out `rclpy.spin()` without realizing it stops the node right after starting. This made it look like the program was broken, when in fact it just exited instantly.

- **Collision when running two nodes with the same name**

When we tried to launch two publishers without changing their node names, it didn't work as expected. Only one was active. We had to use `-ros-args -r __node:=new_name` to give each one a unique name.

Session 2

Preparatory work

In this preparatory work, we deepened our understanding of ROS2 services by creating a mission validation system for a rover. The goal was to implement a server that evaluates mission readiness based on temperature, battery level, and distance, then returns either approval or advice. Unlike the first session where we only used publishers and subscribers, this task introduced a full client-server setup with request-response communication. However, one of the main challenges was creating the custom `.srv` file, which wasn't fully guided. We weren't sure how to structure it, where to place it, or how to properly link it in the rest of the code. We also struggled with getting ROS2 to recognize the new interface, which required multiple rebuilds and careful inspection of the service logic. After some trial and error, we managed to make everything work, gaining a much clearer understanding of how to implement and use custom services in ROS2.

Session

In this second session, we shifted from the basics of ROS2 nodes and communication to visualization, modeling, and simulation. The goal was to build and simulate the Thymio robot using essential tools in the ROS2 ecosystem : Rviz, TFs (transform frames), URDF/Xacro, launch files, and Gazebo.

While the first session focused on understanding how nodes communicate with each other, this one was all about making the robot visible, structured, and interactive. We built a 3D model of the Thymio from scratch using URDF, learned how to use joints to connect parts, and explored how TFs represent spatial relationships in ROS2. Later, we converted our URDF into a Xacro, which made the robot model more flexible and modular.

We then brought the robot to life in Gazebo, where we tested physical interactions, movement, and even added a differential drive plugin and proximity sensors. This session helped us bridge the gap between abstract concepts and a simulated robot that behaves like a real one.

Issues

Just like in Session 1, we ran into a few problems while working on the TP. Some were related to syntax or file structure, while others were linked to URDF modeling or simulation behavior. Here are the main ones we noticed :

- **Incorrect link positioning**

We forgot to properly set the `<origin>` for some links or joints, which caused overlapping parts or floating geometries in Rviz. Rechecking the Rviz visualization and adjusting XYZ/RPY values helped fix this.

- **Missing joints between links**

At one point, we added a second link but forgot to connect it with a joint. The result : Rviz didn't display the link at all. We learned that unconnected links don't appear in the visualization.

- **Hardcoding values in URDF**

When modifying dimensions, some dependent elements (like wheel position) were no longer aligned. This showed the limits of URDF and motivated the switch to Xacro, where we could use variables and math to simplify the model.

- **Wheels not rotating correctly**

We initially forgot to fill in the correct joint names and wheel separation in the plugin configuration. Once fixed, the robot could finally respond to `cmd_vel` commands.

- **Wrong inertia matrix – physics broken in Gazebo**

At one point, we defined `<inertial>` tags manually without using the correct formulas. As a result, the robot's center of mass was off, and it rotated unnaturally or tipped over in Gazebo. Switching to Xacro macros with proper formulas for cylinders and boxes based on mass and dimensions fixed the issue.

- **Robot sliding or moving on its own**

When missing damping/friction values or accurate mass/inertia properties, the robot moved without input after spawning. Adjusting `<dynamics>` and `<inertial>` tags solved this.

- **Step by step construction**

At first, some of us tried to build the thymio all at once which led to multiple errors, difficult to debug. After asking for guidance from the assistance, we started from scratch, step by step.

Session 3

For the thymio mini project, we have submitted 2 .py files and 2 videos and the reason is explained below.

Preparatory work

The task was to create a ROS2 node that controls a turtle in the Turtlesim simulation, making it move in an expanding circular path from the center of the screen while dynamically changing pen color based on the quadrant. The turtle had to stop automatically when reaching the screen boundaries.

We faced several challenges : identifying the window boundaries, tuning linear/angular velocities for a smooth spiral, and detecting quadrant changes correctly. We initially triggered the pen color service too frequently, so we added a check to only call it once per quadrant. To improve boundary detection, we monitored the turtle's pose and added safety margins. We also limited the speed to give time for quadrant detection and pen updates. These fixes made the behavior smoother and the logic more robust. It was a great recap of ROS2 topics, services, and node interaction.

Session

This final session marked the last step of our ROS2 Basics journey. After learning about nodes, topics, services, URDFs, and simulation with Rviz and Gazebo, it was time to put all this knowledge into action.

This project acted as a real integration of everything we had seen so far. We created a controller node that acted as a publisher, continuously sending velocity commands to move the robot forward. At the same time, we used proximity sensor data to stop the robot precisely 5 cm before hitting a wall. Once stopped, our controller interacted with two ROS2 services, one to delete the robot from the simulation and another to spawn it at a new location in the second part of the maze. This added a layer of logic to coordinate real-time robot control with dynamic simulation changes.

By combining movement control (topics) and simulation manipulation (services), this session was a great way to consolidate everything we learned during the course in a hands-on, project-based format.

System Architecture

Each version follows a modular structure centered around a ROS2 node. The robot continuously publishes velocity commands via the `cmd_vel` topic and subscribes to multiple LaserScan topics that simulate the robot's front and rear proximity sensors. An additional subscriber listens to `odom` to retrieve the robot's current orientation, expressed as a yaw angle. This is essential for ensuring accurate turns.

Service clients are used to manage the robot's state in simulation : one for deleting the current robot (`delete_entity`) and another for spawning a new one (`spawn_entity`). Both services are part of the `gazebo_msgs` package. These services allow the robot to disappear from the first zone and reappear with a new pose in the second zone, a critical step in the scenario.

A timer periodically triggers the main control function, which checks sensor values, computes motion commands, and executes logic depending on the robot's situation (turning, stopping, spawning).

Main Logic

Both controllers are using a function that determines how the robot should move based on proximity data. If no obstacle is detected in front, the robot moves forward. As it gets closer to a wall, the speed gradually decreases to ensure a smooth stop.

If the robot is in Zone 1 and reaches a predefined proximity threshold (generally around 5 cm), it triggers a deletion. After a short delay, the robot is respawned in Zone 2 at a new position and orientation. Once in the second maze, the robot continues navigating until a final stopping condition is met.

Turning is handled using a fixed left-turn mechanism. By adding 90 degrees to the current yaw, a new target orientation is set. The robot then rotates left until it reaches this new orientation. This design was chosen for simplicity and consistency, assuming the maze is designed such that left turns are sufficient to navigate corners.

Two Different Stop Strategies

In the first version of the controller, the robot stops based on sensor feedback. More specifically, it uses values from the five front sensors : front-left, center-left, center, center-right, and front-right. If all these sensors detect a very close object (below 7 cm), the system considers this as a clear sign of a wall and stops the robot. While this method is intuitive and flexible, it suffers sometimes from instability. Due to noise or angle variations, sensors may not all trigger simultaneously, causing the robot to miss the stop or to stop prematurely in some rare situations.

In the second version, we took a different approach : instead of relying on sensors, we simply count the number of left turns the robot performs. After five left turns, we consider that the robot has reached the end of the maze and should stop. This method is extremely reliable in a fixed environment, as it guarantees a consistent stopping point. However, it lacks robustness and would fail if the maze had a different layout or number of turns.

Challenges Faced

The main challenge we encountered was defining a robust and reliable stopping condition in Zone 2. At first, we thought sensor-based logic would be enough. However, we quickly realized that sensor readings were unstable and often inconsistent. Depending on the robot's orientation or distance, one or more sensors could return infinite values even when a wall was nearby. This made it difficult to trigger a stop at the right time. We tried tuning the thresholds and comparing multiple sensors, but results were still unreliable.

Another significant challenge was understanding and managing ROS2 services, particularly for deleting and respawning the robot. It wasn't just about calling a function, we had to properly set up the service clients, ensure the service was available before making the request, and carefully manage the asynchronous nature of those calls. This required us to coordinate the timing of the robot's internal logic with the simulation state to avoid overlapping commands or incorrect behavior.

A more subtle but frustrating issue appeared once we started expanding our logic to handle more steps in the maze. Initially, our code worked well to stop in Zone 1 and trigger a transition. However, as we added more logic to handle the full maze traversal, we realized that the robot no longer stopped correctly in Zone 1. Instead, it immediately entered the behavior intended for Zone 2, for instance, starting to turn at every wall. This meant the disjunction between the two phases (before and after the respawn) wasn't working properly. To fix this, we had to rework our state management and introduce additional boolean flags to clearly isolate the logic of each phase.

If we had more time to expand our solution, we would have refined the robot's behavior in the second maze to make it more adaptive and intelligent. For instance, instead of always turning

left when encountering a wall, we could have used the side proximity sensors to dynamically choose whether to turn left or right. This would rely on a differential method comparing the distances provided by the left and right sensors. For example, if the left-side sensors detect a closer wall than the right-side ones, the robot would choose to turn right, and vice versa. This would make navigation more flexible, especially in mazes that do not always favor left-turns.

Regarding the stop condition, we could have explored two options. The first would be to maintain a counter for the number of turns, just as we did, but adjusting it based on the expected layout of the second maze. The second and more robust approach would involve using the proximity sensors to detect a very specific situation where the robot is completely blocked from the front and both sides, or where the left and right distances become nearly symmetrical in a narrow corridor. This could serve as a reliable pattern to indicate the final stopping point, although it would require additional filtering due to sensor instability.

Finally, to improve precision especially in larger mazes where small angular errors can accumulate, we could have integrated a PID controller to smooth out the robot's motion. This would help correct minor orientation drift during forward movements or after rotations, making the navigation smoother and more reliable over longer sequences.

We spent a lot of time trying to make the system robust. We chose to put effort into finding a way to make our simulation independent of the maze, by prioritizing the sensor-based control logic. This led to the lack of time for accomplishing the bonus task. We could have thought of the second maze with the counter method but it wouldn't be very robust.

Conclusion

Throughout the different sessions of this practical, we progressively built our understanding of ROS2, starting from zero experience with its architecture. From basic publishers and subscribers to building our own robot in Gazebo and finally controlling it with real logic, we gained solid hands-on experience that helped demystify the concepts behind robotic systems.

One of the key takeaways was realizing the importance of simulation. Robots are expensive and fragile, and being able to test, debug, and iterate safely in a virtual environment is essential before deploying on real hardware. ROS2 provides exactly that flexibility.

Despite a few challenges, especially with services and sensor behavior, we truly enjoyed the experience and would like to sincerely thank you for your guidance.